

Assignment 1 - 2026

Assignment Spec

Q

^

|

v

1

-

|

+

v

T

Assembler pointers



the assembler should accept exactly one argument. [otherwise, appropriate error message(s) must be directed to STDOUT. In HW-2 we learned about this]



the one argument the assembler can accept, need be a file, should already exist and have the extension .vsc [[Bash Brackets Quick Reference](#) | [assert_not magic?](#) the double square brackets could be useful herein]



[If you are happy with your achievement so far, how about a git add, git commit and a git push] *You can have any number of these. This is just a reminder that we need to use Git repository.*



(Assumption) all the memory locations and registers are initially empty. So any other program apart from addition or subtraction is meaningless.

-
-
- the <filename.vsc> file must follow a proper structure, which is shown in Figure 3 of the spec.
- if the file does not contain any line, then appropriate warning message need be displayed and no .bin file should be produced.
- Otherwise:
 - **Line 1** need be a positive integer - can only be 0 or 2.
 - **if it is 0**, then in the next line no data can be included. Rather it can be only **QUIT,0,0** [we need an exact match - otherwise no bin file should be produced]
 - We want to convert the line **QUIT,0,0** to binary equivalent - the opcode for QUIT is 8 (001000) and then the rest of 12 bits are all zeros. 00100000 (Byte 1) and 00000000 (Byte 2). We need to write them in the file, <filename.bin> [[Bash Brackets Quick Reference](#) | [assert_not magic?](#) the { } curly brace could be useful herein]

For writing binary data to the .bin file:

Byte 1 is 00100000 which is 20 in Hex.

So we simply need to do: `printf '\x20' >> filename.bin`

Similarly we write Byte 2 (after Byte 1 has been written)

We can verify things have been written properly using our **xxd** command.

```
bash-3.2$ bash assembler.sh prog.vsc
It is a QUIT program
*****
The content of the .bin file is:
20
00
```

```
≡ prog.vsc
1      0
2      QUIT,0,0
```

- **if the data at Line 1 is 2,**
 - we are expecting that Line 2 and 3 should be 2 +ve integer numbers - each one in the range [0 -128) - otherwise we need to display appropriate error message and exit.
 - we make use of regular expression ((5) [INFO1112 – Ed Lessons](#)) to verify each one of these two lines are numbers.



we want to use an array for translating an ADD/SUB program. The idea is we keep converting and storing each line and then write everything as a batch. As soon as some thing goes wrong during this conversion process, we abort and do not write out anything i.e., no .bin file gets generated.

In Bash, we can create an empty array as: `dataArray=()`

To assign data e.g., in the first element of this array: `dataArray[0]=$val`



After line 2 and 3 have been validated for, the array should contain 2 elements so far.

The next few lines should begin with either (case sensitive) "LOAD" or "STORE" or "ADD" or "SUB" or "QUIT" or "PRINT" - anything else means it is another error condition - so we would abort (no .bin file would be generated).

We need a loop to process the lines after line 3 (until we hit the QUIT,0,0 line or we reach to a maximum capacity of 100 instructions. This is because the memory capacity is limited and to execute a program in the VSC, the program's .bin version must be loaded into the memory in its entirety.) [Loop was covered in lecture 2 and in tut-3 (4) [INFO1112 – Ed Lessons](#)]

Let's assume Line 4 is: LOAD,3,100

We can see that it is comma (,) separated. So we will use comma (,) as a separator and separate out

LOAD 3 100

But before we aim to do that - what if someone plays foul and have arbitrarily long (incorrect) line? e.g. LOADDDDDDD,0,0 or may be LOAD,000000000000000000000000,128, etc, etc? So we need to devise a strategy.

Let's take some example instructions (to deduce an upper bound).

LOAD,3,255 => length (in terms of characters) is 10

STORE,2,228 => length is 11

ADD,3,193 => length is 9

SUB,3,193 => length is 9

QUIT,0,0 => length is 8

PRINT,2,0 => length is 9

That means the maximum length (in terms of characters) of a valid line is: 11

So any line with length > 11, should be deemed invalid (even before parsing) and we can abort the mission !

[Bash Brackets Quick Reference](#) | [assert_not magic?](#) has example of how to get the length of a variable (string).

✓ Having found a valid (in terms of length) instruction, now we want to parse it.

So if it is LOAD,3,100 at first we do the separation by comma. [can be done in many ways e.g., IFS, awk, sed, etc. But we need to store these 3 pieces into 3 separate variables.]

let's say the variable "ins" contains the instruction: LOAD

the variable "reg" contains the data: 3

the variable "mem" contains the data: 100

⚠ >we check whether "ins" contain anything from the set of "LOAD" or "STORE" or "ADD" or "SUB" or "QUIT" or "PRINT". [hint: grep]

>if yes, then we need to find the opcode for our instruction - in this case it is LOAD, so it is 1 (000001). [hint: since we only have 6 different types of instructions, then the if-elif-else construct may suffice]



>After this we turn our attention to the variable "reg". Valid values for it are 0,1,2,3. Can not be anything else (e.g. a letter, 4, 5, etc). Can not be EMPTY as well. Like an invalid line: ADD -does not provide the reg and mem data.

we can use the integer arithmetic herein. (()) [Bash Brackets Quick Reference](#) | [assert_not magic?](#)



Having found a valid reg data, now we have a byte (6 bits from "ins" and 2 bits from "reg")

we can concatenate 2 variables (strings) in this way: `$opcode$regbin`



>After this we turn our attention to the variable "mem". Valid values for it are [0,255]. Can not be anything else (e.g. a letter, 4, 5, etc). Can not be EMPTY as well. Like an invalid line: ADD,4 -does not provide the reg and mem data. or ADD,4,256 -out of range

```
≡ add.vsc
1      2
2      127
3      12
4      LOAD,0
```

```
[bash-3.2$ bash assembler.sh add.vsc
It is an ADD/SUB program
-----
The mem. part of LOAD,0 is empty.
```



Having found a valid mem data, now we have another byte (a 8-bit representation of the mem data from decimal)

Since we only covered the weighted sum representation like 8 4 2 1, so the dec to bin conversion could be as follows:

```

#weighted sum representation based dec to bin
membin=""
tmp=$mem
for weight in 128 64 32 16 8 4 2 1
do
    if (( $tmp >= $weight )); then
        bit=1
        tmp=$(( $tmp - $weight ))
    else
        bit=0
    fi
    membin="$membin$bit"
    #echo $bit
done

```

We should put this code under a function

<https://edstem.org/au/courses/36385/lessons/109426/slides/810095>

since we have to reuse it multiple times. e.g., to convert the static data that comes before the actual instructions.

Finally, we would write out all the content from the dataArray (as Hex values) to the <filename.bin> file.



After all the hard work, the assembler may output something like the following:

```
[bash-3.2$ bash assembler.sh add.vsc
It is an ADD/SUB program
-----
Line 4: LOAD,0,0 ..... <VALID>
Line 5: ADD,0,1 ..... <VALID>
Line 6: STORE,0,14 ..... <VALID>
Line 7: PRINT,0,0 ..... <VALID>
Found the <QUIT> so ending the conversion procedure .....

*****
Done with the conversion
The content of the .bin file is:
7f
0c
04
00
0c
01
08
0e
24
00
20
00
```

```
≡ add.vsc
1      2
2      127
3      12
4      LOAD,0,0
5      ADD,0,1
6      STORE,0,14
7      PRINT,0,0
8      QUIT,0,0
```

Test cases - Assembler